ODA BULTUM UNIVERSITY
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

GROUP - 1 | THIRD YEAR | FIRST SEMESTER

STUDENT ATTENDANCE MANAGEMENT SYSTEM

JAVA PROGRAMMING PROJECT REPORT

GROUP MEMBERS

No.	Student Name	ID Number
1	Leta Girma	_____
2	Lema Gezu	_____
3	Ahmed Baba	_____
4	Abraham	_____
5	Ana Kadi	_____
6	_____	_____

Academic Year	2026
Year of Study	Third Year
Semester	First Semester
Project Group	Group - 1

Submitted to the Department of Electrical and Computer Engineering, Oda Bultum University.

ABSTRACT

The Student Attendance Management System is a Java console application developed to help manage student attendance. The system allows an operator to register students, record whether each student is present or absent for a class session, and display an attendance report. It calculates each student's attendance percentage automatically. The project is implemented using three classes: Student, AttendanceManager, and Main. It demonstrates object-oriented programming concepts such as encapsulation, classes, objects, and separation of responsibilities. Student records are stored in an ArrayList while the program is running.

1. INTRODUCTION

Attendance is an important part of academic administration. Keeping accurate records helps instructors monitor participation and understand each student's attendance history. Recording attendance manually and calculating percentages can take time. This project presents a simple computerized system that performs these basic tasks through a menu-driven Java program.

The application is designed for use in a terminal or command prompt. It provides options to add students, mark attendance, display a report, and exit the program.

2. PROBLEM STATEMENT

Manual attendance recording may involve repetitive work and can make it difficult to calculate attendance percentages consistently. Preparing a summary for several students can also take time. The proposed system addresses these basic challenges by storing student details, checking for duplicate student IDs, recording attendance counts, and calculating percentages automatically.

3. OBJECTIVES AND SCOPE

3.1 General objective

To develop a simple Java-based Student Attendance Management System for registering students, recording attendance, and preparing attendance summaries.

3.2 Specific objectives

• To register students using their identification numbers and full names.

• To prevent duplicate student IDs from being registered.

• To record each student's attendance as present or absent.

• To calculate the number of attended classes and total recorded classes.

• To calculate and display each student's attendance percentage.

• To apply basic object-oriented programming concepts in Java.

3.3 Scope

The current system supports student registration, attendance marking for all registered students, and report display during one program session. It is a basic console-based project. It does not currently save records permanently or maintain attendance dates.

4. SYSTEM REQUIREMENTS

4.1 Hardware requirements

• Desktop or laptop computer.

• Keyboard and monitor.

• Sufficient memory and storage to run a Java development environment.

4.2 Software requirements

• Java Development Kit (JDK).

• Java IDE or text editor such as IntelliJ IDEA, Eclipse, or Visual Studio Code.

• Terminal or command prompt.

4.3 Functional requirements

Code	Functional requirement
FR-01	The system shall register a student with an ID and name.
FR-02	The system shall reject a duplicate student ID.
FR-03	The system shall allow attendance to be marked for registered students.
FR-04	The system shall update present and total class counts.
FR-05	The system shall calculate and display attendance percentages.
FR-06	The system shall allow the user to exit the application.

5. SYSTEM DESIGN AND ARCHITECTURE

5.1 Class descriptions

Class	Responsibilities
Student	Stores student ID, name, present count, and total classes. Provides methods to update attendance and calculate
AttendanceManager	Maintains the student ArrayList and handles registration, attendance marking, and report display.
Main	Runs the application, displays the menu, receives user input, and calls the manager methods.

5.2 Data structure

The program uses ArrayList<Student> to store Student objects. The list can grow as new students are registered. The data remains in memory only while the program is running.

5.3 Attendance calculation

The system stores two values for each student: presentCount and totalClasses. Every attendance entry increases totalClasses by one. If the student is present, presentCount also increases by one.

Attendance Percentage = (Present Count / Total Classes) × 100

If totalClasses is zero, the method returns 0.0 to avoid division by zero. The program converts presentCount to a double before division so that the result can include decimal values.

6. OBJECT-ORIENTED PROGRAMMING CONCEPTS

6.1 Encapsulation

The fields in the Student class are declared private. This prevents other classes from directly changing student data. Public methods such as getId(), getName(), markAttendance(), and getAttendancePercentage() provide controlled access.

6.2 Classes and objects

Student, AttendanceManager, and Main are classes. When a student is registered, the program creates an object using new Student(id, name). Each Student object maintains its own attendance counts.

6.3 Abstraction and separation of responsibilities

AttendanceManager provides methods for registration, attendance marking, and report generation. Main handles user interaction. Separating these responsibilities makes the code easier to understand and maintain.

6.4 Java collection

ArrayList is a Java collection used to store a variable number of Student objects. The program uses for-each loops to search and process the students in the list.

7. PROGRAM OPERATION

1. Start application: The Main class creates a Scanner and an AttendanceManager object.

2. Display menu: The user sees four options: add a student, mark attendance, display report, and exit.

3. Add student: The user enters the student's ID and full name. The manager checks for a duplicate ID before registering the student.

4. Mark attendance: The program asks for p (present) or a (absent) for each registered student.

5. Display report: The system prints each student's ID, name, present/total count, and attendance percentage.

6. Exit: The program ends the menu loop and closes the Scanner.

8. TESTING AND EXPECTED RESULTS

The following test cases describe the expected behavior of the supplied program. The group should run these tests in a Java environment and record actual results before final submission.

Test	Condition / input	Expected result
TC-01	Register a new student	Student is added successfully.
TC-02	Register an existing ID again	Duplicate ID message appears; no second record is added.
TC-03	Mark attendance with no students	System asks the user to register students first.
TC-04	Enter p for a student	Present count and total class count both increase.
TC-05	Enter a for a student	Only total class count increases.
TC-06	Display report	Student attendance counts and percentage are displayed.
TC-07	Choose option 4	The application exits.

8.1 Example calculation

If a student attends 8 out of 10 recorded classes, the attendance percentage is (8 ÷ 10) × 100 = 80.00%. The displayInfo() method formats the percentage to two decimal places.

9. LIMITATIONS AND FUTURE IMPROVEMENTS

9.1 Current limitations

• Student records are stored in memory and are lost when the program closes.

• The program does not store the date of each attendance session.

• Any input other than p is treated as absent; attendance input is not fully validated.

• Non-numeric menu input may cause an input exception because nextInt() is used.

• The system does not currently support editing, deleting, or searching student records.

9.2 Future improvements

• Add validation so that only p or a is accepted for attendance.

• Handle invalid menu input safely.

• Save student records using files or a database.

• Store attendance dates and prepare reports for selected date ranges.

• Add search, update, and delete functions.

• Develop a graphical user interface and allow report export.

10. CONCLUSION

The Student Attendance Management System is a basic Java application that supports student registration, duplicate ID checking, attendance recording, and attendance percentage calculation. Its three-class structure demonstrates important object-oriented programming concepts, especially encapsulation and separation of responsibilities. Although the current version stores data only during program execution, it provides a useful foundation for future improvements such as permanent storage, input validation, dated attendance records, and a graphical interface.

APPENDIX A: JAVA SOURCE CODE

The source code provided by the group is included below. Save the program in a file named **Main.java**, because Main is the public class.

```
import java.util.ArrayList;
import java.util.Scanner;
// 1. Student Class (Encapsulation and OOP Concept)
class Student {
    private String id;
    private String name;
    private int presentCount;
    private int totalClasses;
    public Student(String id, String name) {
        this.id = id;
        this.name = name;
        this.presentCount = 0;
        this.totalClasses = 0;
    }
    public String getId() {
        return id;
    }
    public String getName() {
        return name;
    }
    public void markAttendance(boolean isPresent) {
        totalClasses++;
        if (isPresent) {
            presentCount++;
        }
    }
    public double getAttendancePercentage() {
        if (totalClasses == 0) return 0.0;
        return ((double) presentCount / totalClasses) * 100;
    }
    public void displayInfo() {
        System.out.printf("ID: %-10s | Name: %-20s | Attendance: %d/%d (%.2f%%)\n",
                id, name, presentCount, totalClasses, getAttendancePercentage());
    }
}
// 2. AttendanceManager Class (Business Logic)
class AttendanceManager {
    private ArrayList<Student> studentList;
    public AttendanceManager() {
        studentList = new ArrayList<>();
    }
    // Register a new student
    public void addStudent(String id, String name) {
        for (Student s : studentList) {
            if (s.getId().equalsIgnoreCase(id)) {
                System.out.println("Error: A student with this ID already exists!");
                return;
            }
        }
        studentList.add(new Student(id, name));
        System.out.println("Student registered successfully!");
    }
    // Mark attendance for students
    public void markAttendance(Scanner scanner) {
        if (studentList.isEmpty()) {
            System.out.println("No students registered yet! Please add students first.");
            return;
        }
        System.out.println("\n--- MARK ATTENDANCE (Present: p / Absent: a) ---");
        for (Student s : studentList) {
            System.out.print("Student: " + s.getName() + " (" + s.getId() + ") [p/a]: ");
            String input = scanner.next();
            boolean isPresent = input.equalsIgnoreCase("p");
            s.markAttendance(isPresent);
        }
```

```
        System.out.println("Today's attendance marked successfully!");
    }
    // Display attendance report
    public void displayReport() {
        if (studentList.isEmpty()) {
            System.out.println("No students registered yet!");
            return;
        }
        System.out.println("\n================================================================");
        System.out.println("     ODA BULTUM UNIVERSITY - STUDENT ATTENDANCE REPORT");
        System.out.println("================================================================");
        for (Student s : studentList) {
            s.displayInfo();
        }
        System.out.println("================================================================\n");
    }
}
// 3. Main Class (Program Execution & Menu)
public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        AttendanceManager manager = new AttendanceManager();
        boolean exit = false;
        System.out.println("*****************************************************************");
        System.out.println("    WELCOME TO ODA BULTUM UNIVERSITY ATTENDANCE SYSTEM");
        System.out.println("*****************************************************************");
        while (!exit) {
            System.out.println("\nAVAILABLE OPTIONS:");
            System.out.println("1. Add New Student");
            System.out.println("2. Mark Attendance");
            System.out.println("3. Display Attendance Report");
            System.out.println("4. Exit");
            System.out.print("Enter your choice (1-4): ");
            int choice = scanner.nextInt();
            scanner.nextLine(); // consume newline
            switch (choice) {
                case 1:
                    System.out.print("Enter Student ID (e.g., OBU/001/18): ");
                    String id = scanner.nextLine();
                    System.out.print("Enter Student Full Name: ");
                    String name = scanner.nextLine();
                    manager.addStudent(id, name);
                    break;
                case 2:
                    manager.markAttendance(scanner);
                    break;
                case 3:
                    manager.displayReport();
                    break;
                case 4:
                    exit = true;
                    System.out.println("Thank you! Exited the system successfully.");
                    break;
                default:
                    System.out.println("Invalid choice! Please choose a number between 1 and 4.");
            }
        }
        scanner.close();
    }
}
```